

Nominal State-Separating Proofs

Markus Krabbe Larsen
Theoretical Computer Science
IT University of Copenhagen
Copenhagen, Denmark
krml@itu.dk

Carsten Schürmann
Theoretical Computer Science
IT University of Copenhagen
Copenhagen, Denmark
carsten@itu.dk

Abstract—State-separating proofs are a powerful tool to structure cryptographic arguments, so that they are amenable for mechanization, as has been shown through implementations, such as SSProve. However, the treatment of separation for heaps has never been satisfactorily addressed. In this work, we present the first comprehensive treatment of nominal state separation in state-separating proofs using nominal sets. We provide a Rocq library, called Nominal-SSProve, that builds on nominal state separation supporting mechanized proofs that appear more concise and arguably more elegant.

I. INTRODUCTION

State-separating proofs [6] have become a widely accepted tool to express cryptographic games and reductions in the computational model in a formal and precise way to make them palatable for modern verification tools and hereby increase the overall quality of the arguments through formal verification. The central idea in state-separating proofs is to express games, reductions, and the adversary as stateful *packages* that can be combined in modular ways to describe cryptographic security proofs. The state is used to store secret information local to each package.

As an example, consider the Diffie-Hellman key exchange protocol that stores a random value created during the preparation of the first message in the package’s local state, and then accesses it again to compute a shared secret in a subsequent message. If an adversary had access to the secret information stored within the state, he would with probability 1 be able to win the cryptographic game, which would render the way cryptographic proofs are done void and meaningless. Thus, the challenge is to protect the state of each package from accidental or malicious access by other packages, for example, by sharing state variable names.

The original formulation of state-separating proofs [6] does not address this challenge adequately, but refers instead to informal on-demand tacit renaming of state variables when packages are composed in order to prevent state variable capture. Similarly, extending EasyCrypt by incorporating state-separating proofs [8] does not address the challenge either but leaves it instead to future work. Implementations such as SSProve [13] (in Rocq [21]) circumvent this challenge through ad-hoc assumptions about the disjointness of state variables in packages at the expense of modularity.

In this paper, we introduce *nominal state-separating proofs*, extending state-separating proofs by nominal sets [10] to enforce state separation between packages. In nominal state-separating proofs, packages modeling adversaries do not have access to the state of other packages by construction, as each package has its own local state name space. When combining packages, nominal sets ensure that state variables are automatically renamed away from each other so that state variable capture becomes impossible. We demonstrate nominal state-separating proofs by means of showing that the *ElGamal cryptosystem*[9] satisfies the *public key one time secrecy* security property following the proof in [17, Chapter 15.3].

To demonstrate the power of nominal state-separating proofs, we have implemented them in a library that extends SSProve, called Nominal-SSProve.¹ With this library, operations for combining nominal packages – or *modules* as we call them – are semantically transparent, which means that a user does not have to worry about state variable capture, even when quantifying over adversaries. A fully mechanized proof of the running example in Nominal-SSProve can be found here.²

We now expand on the challenge and pinpoint the lack of modularity of state-separating proofs for game hopping and cryptographic reductions. The main idea behind state-separating proofs is to reason about the indistinguishability of a pair of games G_1 and G_2 by an adversary $\mathcal{A}$. We may find the two games to be perfectly indistinguishable, meaning that the adversary cannot distinguish between them, written as $\text{Adv}_{G_1, G_2}^{\#}(\mathcal{A}) = 0$, where the $\#$ symbol represents a non-nominal advantage definition. We must assume (and make explicit) that the state variables accessible by the adversary are disjoint from the state variables used to define the two games, written as $\mathcal{A} \# G_1$ and $\mathcal{A} \# G_2$. *Without nominals, the only way to represent disjointness constraints is to make them explicit.*

Let G_1 , G_2 , and G_3 be three games, where the pairs of (G_1, G_2) and (G_2, G_3) are perfectly indistinguishable. To show that (G_1, G_3) are perfectly indistinguishable, we have to prove

¹<https://github.com/MarkusKL/nominal-ssprove/tree/v1.0.0>

²<https://github.com/MarkusKL/nominal-ssprove/tree/v1.0.0/theories/Example/PK>

that,

$$\begin{aligned}\forall \mathcal{A}. \mathcal{A} \# G_1 \wedge \mathcal{A} \# G_2 &\supset \text{Adv}_{G_1, G_2}^\#(\mathcal{A}) = 0, \\ \forall \mathcal{A}. \mathcal{A} \# G_2 \wedge \mathcal{A} \# G_3 &\supset \text{Adv}_{G_2, G_3}^\#(\mathcal{A}) = 0 \\ \vdash \forall \mathcal{A}. \mathcal{A} \# G_1 \wedge \mathcal{A} \# G_3 &\supset \text{Adv}_{G_1, G_3}^\#(\mathcal{A}) = 0\end{aligned}$$

assuming that the triangle inequality

$$\text{Adv}_{G_1, G_2}^\#(\mathcal{A}) + \text{Adv}_{G_2, G_3}^\#(\mathcal{A}) \geq \text{Adv}_{G_1, G_3}^\#(\mathcal{A})$$

holds. This, however, appears to be impossible, since there is no assumption $\mathcal{A} \# G_2$ in the consequent of the entailment above. We cannot be sure that the adversary's state variables are disjoint from those of game G_2 . Making the assumption $\mathcal{A} \# G_2$ explicit is possible, but it does not generalize well.

Our solution is to revert to nominal state-separating proofs instead, which provides a definition of *nominal advantage* $\text{Adv}_{G_1, G_2}(\mathcal{A})$ that enforces state separation between games (G_1, G_2) and the adversary $\mathcal{A}$. As a consequence, there is no longer a need for explicit disjointness assumptions. Therefore, perfect indistinguishability of G_1 and G_3 follows directly from the triangle inequality, as we show in Corollary 37.

$$\begin{aligned}\forall \mathcal{A}. \text{Adv}_{G_1, G_2}(\mathcal{A}) &= 0, \\ \forall \mathcal{A}. \text{Adv}_{G_2, G_3}(\mathcal{A}) &= 0 \\ \vdash \forall \mathcal{A}. \text{Adv}_{G_1, G_3}(\mathcal{A}) &= 0.\end{aligned}$$

A. Contributions

- 1) We make precise the informal requirement of tacit variable renaming in [6].
- 2) We solve the challenge of state variable capture in state-separating proofs.
- 3) We define a formal semantics of nominal state-separating proofs.
- 4) We implement Nominal-SSProve as a demonstrator in Rocq.
- 5) We mechanize the reduction from public key one time secrecy for ElGamal to Decisional Diffie–Hellman (DDH). As a result we identify two mistakes in the mechanization of the reduction in SSProve [13].

B. Related Work

State-separating proofs are built on the perfect indistinguishability of games, which is captured by a program logic, here probabilistic Hoare logic [4]. The extension of state-separating proofs by nominal sets in our work is orthogonal to the choice the program logic: probabilistic Hoare logic, for example, is about reasoning about code and not modules and is therefore agnostic to whether or not variables have been renamed. Nominal-SSProve builds upon the same probabilistic relational Hoare logic as SSProve – no modifications are necessary.

The Clutch system [11] derives its power directly from separation logic [16] combined with an elegant treatment of asynchronous couplings for modeling probabilistic arguments in cryptography. So far, Clutch has not been used to model

state-separating proofs. It can give, however, automatic disjointness guarantees simply based on the fact that it is variant of separation logic. Recently, the program logic Approxis [12] was proposed for reasoning about cryptographic proofs with error bounds. In contrast, in Nominal-SSProve, we reason only about perfect indistinguishability. Both systems, Clutch and Approxis are designed as program logics supporting features such as higher-order store, random sampling and asynchronous couplings and others. We emphasize that the objective with this paper is not to derive a better and stronger program logic, but to ensure that Nominal-SSProve provides a good compromise between expressiveness and ease of use.

There are also other recent advances in program logic research, for example, conditional contextual refinement [18], refinement composition logic [19], and quantitative refinement logic [14]. These papers provide useful new ideas that could strengthen the expressiveness of state-separating proofs but may not necessarily improve the ease of use. We will investigate the connection to Nominal-SSProve in future work.

How to handle variable names and α -conversion has been studied by many. We refer the reader to the POPLmark challenge [2] for an overview.

State-separating proofs have been used to express the proof of key-schedule security for the TLS1.3 standard on paper [5], and many other examples in the style of state-separating proofs can be found in [17].

C. Overview

This paper is organized as follows. In Section II we define a probabilistic stateful language together with a module system for capturing state-separating proofs. Its static and operational semantics are also described. In Section III, we review the theory of nominal sets and establish that our notion of heap forms a nominal set. In the following Section IV we then show that our notion of adversarial advantage is compatible with nominal sets. In Section V we give a brief overview of Nominal-SSProve and discuss the mistakes we identified in the development of ElGamal in SSProve. We then conclude in Section VI and assess results.

II. A LANGUAGE FOR STATE-SEPARATING PROOFS

We turn our attention to the definition of a simple formal language to express state-separating proofs that extends the simply typed λ -calculus with product types, sum types, a static heap, sampling, and a simple module system that adequately represents sequential and parallel package composition following [6]. The language is inspired by functional programming. It allows us to prove correct the nominal constructions that we shall introduce in Section IV in a proof-assistant-agnostic way. Packages are encoded as modules and package composition as module composition. Existing tools such as EasyCrypt and Rocq/SSProve embed this language into higher-order logic and the calculus of inductive constructions, respectively, and inherit additional language features that are irrelevant for this presentation. In Section II-B we will cover syntactic categories and in Section II-C the static semantics. As the language is

probabilistic, we introduce the operational semantics and the sub-distribution monad in Sections II-D and II-E, respectively. Finally, in Section II-F we introduce the algebraic equations, that the module system obeys.

A. Base Types and Types

Central to our design is the notion of a sample. As in related work, we capture sample spaces by finite sets, except here, we approximate finite sets proof-theoretically.

$$\alpha \in \text{Base} ::= \text{unit} \mid \alpha_1 \times \alpha_2 \mid \alpha_1 + \alpha_2$$

Base types model finite sets to draw samples from: the unit type is inhabited by one element (). Products concatenate samples, and sums capture non-deterministic choice. A bit, for example, may be represented as $\text{bit} = \text{unit} + \text{unit}$, which contains two elements $\text{inl}()$ and $\text{inr}()$. A sample space of size n may be expressed by $\text{fin } n$, defined as follows:

$$\begin{aligned} \text{fin } 1 &= \text{unit} \\ \text{fin } (2n) &= \text{bit} \times \text{fin } (n) \\ \text{fin } (2n + 1) &= \text{bit} \times \text{fin } (n) + \text{unit}. \end{aligned}$$

Given a security parameter λ , we select a group G with a cyclic subgroup of order $q \geq 2^\lambda$ generated by $g \in G$. Formally, G and the subgroup generated by g are denoted by base type $\text{el} = \text{fin } q$ and $\text{exp} = \text{fin } q$, respectively. We define the cardinality $|\alpha|$ of base type α as follows.

$$|\text{unit}| = 1 \quad |\alpha_1 \times \alpha_2| = |\alpha_1| \cdot |\alpha_2| \quad |\alpha_1 + \alpha_2| = |\alpha_1| + |\alpha_2|$$

By induction on α , it is easy to see that any base type α is inhabited.

Theorem 1 (Inhabitation of base types). *Let α be a base type, then α is inhabited.*

Finally, we declare function types

$$\tau \in \text{Type} ::= \alpha \mid \tau_1 \rightarrow \tau_2$$

used to type cryptographic algorithms.

B. Values, Expressions, and Modules

Base types and types are inhabited by values which are computed by expressions.

$$\begin{aligned} v \in \text{Val} &::= () \mid \text{rec } f \ x = e \\ &\mid (v_1, v_2) \mid \text{inl}(v) \mid \text{inr}(v) \\ e \in \text{Exp} &::= x \mid v \mid c \mid e_1 \ e_2 \mid \text{fst}(e) \mid \text{snd}(e) \\ &\mid \text{case } e \text{ of } \text{inl}(x_1) \Rightarrow e_1 \\ &\quad \mid \text{inr}(x_2) \Rightarrow e_2 \\ &\mid !a \mid a := e \mid \text{sample}(\alpha) \mid F(e) \end{aligned}$$

Most of the constructs are self-explanatory, we only explain those that are not. $x, f \in \mathbb{V}$ are local variables, subject to instantiation by substitution. State variables are represented by atoms and allow us to reference base values stored on the heap.

Definition 1 (Atoms). *The set $\mathbb{A}$ defines a countably infinite set of state variables denoted $a_1, a_2, a_3 \dots$. In this paper, we use atom and state variable interchangeably.*

The set of atoms is infinite, so that in any context of finitely many atoms it is possible to pick a new atom. The nominal constructions that we propose in Section III are built on these atoms.

Finally, $F \in \mathbb{I}$ are identifiers referring to expressions implemented in other modules, which we define below. The **rec** construct serves as both the fix-point constructor and abstraction according to [13]. The two concepts could have been separated as in other presentations, but nothing is to be gained in our setting. $!a$ resolves an atom $a \in \mathbb{A}$. The expression $a := e$ binds the result sample of e to state variable a in the heap. There is an expression to sample uniformly from a base type α of size $|\alpha|$, for which we write $\text{sample}(\alpha)$. Finally, the expression $F(e)$ refers to a call of $F \in \mathbb{I}$ with argument e . We use parentheses in this case in order to distinguish it from expression application $e_1 \ e_2$.

We define a few shorthands that will prove useful when writing out expressions in the language. Note that we encode failure by looping indefinitely, and that we omit () when the value is already enclosed by parentheses. Finally, we allow () in place of a binding occurrence of a local variable, when that is the only possible value.

$$\begin{aligned} \text{bool} &= \text{unit} + \text{unit} \\ \text{false} &= \text{inl}() \\ \text{true} &= \text{inr}() \\ \text{fail} &= (\text{rec } f \ x = f \ x) () \\ \lambda x. e &= \text{rec } f \ x = e \\ \text{let } x = e_1 \text{ in } e_2 &= (\lambda x. e_2) \ e_1 \\ \text{let } \text{inl}(x) = e_1 \text{ in } e_2 &= \text{case } e_1 \text{ of } \text{inl}(x) \Rightarrow e_2 \\ &\quad \mid \text{inr}(y) \Rightarrow \text{fail} \\ \text{let } \text{inr}(x) = e_1 \text{ in } e_2 &= \text{case } e_1 \text{ of } \text{inl}(y) \Rightarrow \text{fail} \\ &\quad \mid \text{inr}(x) \Rightarrow e_2 \\ a := e_1 ; e_2 &= (\lambda (). e_2) (a := e_1) \end{aligned}$$

To explain the mechanism behind identifier resolution, we introduce modules next.

$$M \in \text{Module} ::= \text{module} \mid M \text{ fun } F \ x = e$$

A *module* groups together the functionality of an application, for example, a cryptosystem, a game, an oracle, or a reduction. Modules are called packages in [6]. Interfaces declare the types of the respective identifiers of a module. We distinguish between import and export interfaces, for which we write I and E , respectively; however, this cannot be observed in all cases, as interfaces may appear both in the import and export position.

C. Static Semantics

Given a fixed heap type for atoms Σ and import interface I for cryptographic algorithms that may be invoked, a set of constants Δ , and a context Γ ,

$$\begin{aligned} \Sigma \in \text{Heap} &::= \cdot \mid \Sigma, a : \alpha \\ \Gamma \in \text{Ctx} &::= \cdot \mid \Gamma, x : \tau \\ \Delta \in \text{Const} &::= \cdot \mid \Delta, c : \tau \\ I \in \text{Interface} &::= \text{interface} \mid I \text{ sig } F : \alpha_1 \rightarrow \alpha_2. \end{aligned}$$

In the case of a non-empty *Heap*, *Ctx*, or *Const*, we omit the leading $\cdot$.

Example 1. In our running example, constants are group operations. Formally, we write $\text{mult}(x, y)$ for $x \cdot y$, $\text{pow}(g, x)$ for g^x and $\text{powinv}(g, x)$ for g^{-x} . They are declared as follows.

$$\begin{aligned} \Delta &= \text{mult} && : \text{el} \times \text{el} \rightarrow \text{el}, \\ &\text{pow} && : \text{el} \times \text{exp} \rightarrow \text{el}, \\ &\text{powinv} && : \text{el} \times \text{exp} \rightarrow \text{el} \end{aligned}$$

We define the typing judgments for expressions e in context Γ as follows: $\Sigma; I \mid \Gamma \vdash_{\Delta} e : \tau$. Since Δ is always fixed throughout an argument involving state-separating proofs, we omit it from the judgment and write henceforth $\Sigma; I \mid \Gamma \vdash e : \tau$. The rules for this judgment are given in Figure 1. All but the last six are standard. In the interest of brevity, we omit the definition of well-formedness judgments for *Heap*, *Ctx*, *Const*, and *Interface* but we remark that they are implicitly required in rules *unit*, *ax*, *deref*, *sample*, and *module*.

Figure 1 also introduces the typing rules for well-typed modules. Informally, module M is well-typed of export interface E given a heap typing for Σ an import interface I , and a set of constants Δ , written as judgment $\Sigma; I \vdash_{\Delta} M : E$ and defined by rules *module* and *fun*. For reasons mentioned above, we omit the Δ from this judgment, and simply write $\Sigma; I \vdash M : E$.

Example 2. We define the standard three algorithms for the ElGamal cryptosystem as abbreviations in Figure 2.

Example 3. We define the modules necessary to capture the DDH assumption in Figure 3. In our example, we use a lazy form of randomness as captured by the two atoms $\text{mga} \in \mathbb{A}$ and $\text{init} \in \mathbb{A}$. $\text{GETA} \in \mathbb{I}$ and $\text{GETBC} \in \mathbb{I}$ are identifiers. We leave it to the reader to verify that the modules are well-typed, i.e.

$$\begin{aligned} \text{mga} : \text{unit} + \text{el}; \cdot &\vdash \text{DDH}^0 : \text{I-DDH} \\ \text{init} : \text{unit} + \text{unit}; \cdot &\vdash \text{DDH}^1 : \text{I-DDH}. \end{aligned}$$

For the rest of the paper it is useful to define $\text{DDH} = (\text{DDH}^0, \text{DDH}^1)$, also referred to as a game pair in [6].

For our final example in this section, we define the interface for adversaries that we use throughout this paper.

Definition 2 (Interface of the Adversary). An adversary interface is defined as

$$\text{I-ADV} = \text{interface sig RUN} : \text{unit} \rightarrow \text{bool}$$

In summary, we have presented a simple functional language that is powerful enough to express cryptographic algorithms. If anything, this language is too powerful, because general recursion captures a class of adversaries beyond probabilistic, polynomial time computable functions. In Section IV, we show that values and expressions form nominal sets. In future work, we might consider extending base types to infinite types, which would complicate working with probability distributions but may offer other benefits, such as greater expressiveness. Another way to extend this language is to generalize base types beyond the sample space, as done, for example, in [11].

D. Operational Semantics

We have chosen to give call-by-value probabilistic operational semantics to our language. It might be possible to experiment with other calling conventions, which we leave to future work. The operational semantics are defined as a small step relation that transforms configurations *Cfg* consisting of an expression e and the current state of the heap σ .

$$\sigma \in \text{State} ::= \cdot \mid \sigma, a := v$$

As a judgment we write $\langle e; \sigma \rangle \rightarrow_p \langle e'; \sigma' \rangle$ for a single step, where p is a probability $0 < p \leq 1$, with which this step is taken. For most configurations there is only one deterministic next step the interpreter can take, meaning that $p = 1$, written as $\langle e; \sigma \rangle \rightarrow \langle e'; \sigma' \rangle$. For sampling, there might be many possible next steps, albeit with a total probability adding to one. There is no step rule for $F(e)$, since calls to an identifier will have been replaced by the corresponding inlined expression during module composition. In the interest of space, we introduce in Figure 4 only the essential reductions and leave the congruence rules to the imagination of the reader. Note that the probabilities of each essential reduction are carried through the congruence closure.

By induction on heap type Σ appealing to Theorem 1, we can show that

Theorem 2. For all heap types Σ , there exists a state σ that matches Σ .

E. Sub-distribution Monad

Although the operational semantics is probabilistic, a terminating computation will always result in one of finitely many configurations, which means that we can use a sub-distribution monad [11] to capture the probability for each such final configuration.

Definition 3 (Sub-distribution monad). Let $\mathcal{D}(X)$ be the discrete sub-distribution over X consisting of functions $p : X \rightarrow [0, 1]$ where $\sum_{x \in X} p(x) \leq 1$ with $\text{unit} : X \rightarrow \mathcal{D}(X)$, $\text{bind} : \mathcal{D}(X) \rightarrow (X \rightarrow \mathcal{D}(Y)) \rightarrow \mathcal{D}(Y)$, and $\text{zero} : \mathcal{D}(X)$.

$$\begin{aligned} \text{unit}(x)(x') &= \begin{cases} 1 & \text{if } x = x' \\ 0 & \text{otherwise} \end{cases} \\ \text{bind}(p, f)(y) &= \sum_{x \in X} p(x) \cdot f(x)(y) \\ \text{zero}(x) &= 0 \end{aligned}$$

The step derivation induces a probability distribution on configurations which we capture formally as the function $\text{step} : \text{Cfg} \rightarrow \mathcal{D}(\text{Cfg})$ given by

$$\text{step}(e, \sigma)(e', \sigma') = \begin{cases} p & \text{if } \langle e, \sigma \rangle \rightarrow_p \langle e', \sigma' \rangle \\ 0 & \text{otherwise} \end{cases}$$

The iterated distribution of executing $n \in \mathbb{N}$ steps results in a probability distribution of values by adding the probabilities

$$\begin{array}{c}
\frac{}{\Sigma; I \mid \Gamma \vdash () : \text{unit}} \text{unit} \quad \frac{\Gamma(x) = \tau}{\Sigma; I \mid \Gamma \vdash x : \tau} \text{ax} \\
\\
\frac{\Sigma; I \mid \Gamma, f : \tau_1 \rightarrow \tau_2, x : \tau_1 \vdash e : \tau_2}{\Sigma; I \mid \Gamma \vdash \mathbf{rec} \ f \ x = e : \tau_1 \rightarrow \tau_2} \text{rec} \quad \frac{\Sigma; I \mid \Gamma \vdash e_1 : \tau_2 \rightarrow \tau_1 \quad \Sigma; I \mid \Gamma \vdash e_2 : \tau_2}{\Sigma; I \mid \Gamma \vdash e_1 \ e_2 : \tau_1} \text{app} \\
\\
\frac{\Sigma; I \mid \Gamma \vdash e_1 : \alpha_1 \quad \Sigma; I \mid \Gamma \vdash e_2 : \alpha_2}{\Sigma; I \mid \Gamma \vdash (e_1, e_2) : \alpha_1 \times \alpha_2} \text{pair} \quad \frac{\Sigma; I \mid \Gamma \vdash e : \alpha_1 \times \alpha_2}{\Sigma; I \mid \Gamma \vdash \mathbf{fst}(e) : \alpha_1} \text{fst} \quad \frac{\Sigma; I \mid \Gamma \vdash e : \alpha_1 \times \alpha_2}{\Sigma; I \mid \Gamma \vdash \mathbf{snd}(e) : \alpha_2} \text{snd} \\
\\
\frac{\Sigma; I \mid \Gamma \vdash e : \alpha_1}{\Sigma; I \mid \Gamma \vdash \mathbf{inl}(e) : \alpha_1 + \alpha_2} \text{inl} \quad \frac{\Sigma; I \mid \Gamma \vdash e : \alpha_2}{\Sigma; I \mid \Gamma \vdash \mathbf{inr}(e) : \alpha_1 + \alpha_2} \text{inr} \\
\\
\frac{\Sigma; I \mid \Gamma \vdash e : \alpha_1 + \alpha_2 \quad \Sigma; I \mid \Gamma, x_1 : \alpha_1 \vdash e_1 : \tau \quad \Sigma; I \mid \Gamma, x_2 : \alpha_2 \vdash e_2 : \tau}{\Sigma; I \mid \Gamma \vdash \mathbf{case} \ e \ \mathbf{of} \ x_1 \Rightarrow e_1 \mid x_2 \Rightarrow e_2 : \tau} \text{case} \\
\\
\frac{\Sigma(a) = \alpha}{\Sigma; I \mid \Gamma \vdash !a : \alpha} \text{deref} \quad \frac{\Sigma(a) = \alpha \quad \Sigma; I \mid \Gamma \vdash e : \alpha}{\Sigma; I \mid \Gamma \vdash a := e : \text{unit}} \text{assign} \\
\\
\frac{I(F) = \alpha_1 \rightarrow \alpha_2 \quad \Sigma; I \mid \Gamma \vdash e : \alpha_1}{\Sigma; I \mid \Gamma \vdash F(e) : \alpha_2} \text{call} \quad \frac{}{\Sigma; I \mid \Gamma \vdash \mathbf{sample}(\alpha) : \alpha} \text{sample} \\
\\
\frac{}{\Sigma; I \vdash \mathbf{module} : \mathbf{interface}} \text{module} \quad \frac{\Sigma; I \vdash M : E \quad \Sigma; I \mid x : \alpha_1 \vdash e : \alpha_2}{\Sigma; I \vdash (M \ \mathbf{fun} \ x \ F = e) : (E \ \mathbf{sig} \ F : \alpha_1 \rightarrow \alpha_2)} \text{fun}
\end{array}$$

Fig. 1. Static Semantics

$$\begin{array}{lll}
\text{keygen} : \text{exp} \times \text{el} & \text{enc} : \text{el} \rightarrow \text{el} \rightarrow (\text{el} \times \text{el}) & \text{dec} : \text{exp} \rightarrow (\text{el} \times \text{el}) \rightarrow \text{el} \\
\text{keygen} = & \text{enc} = \lambda \text{pk}. \lambda \text{m}. & \text{dec} = \lambda \text{sk}. \lambda \text{c}. \\
\quad \mathbf{let} \ sk = \mathbf{sample}(\text{exp}); & \quad \mathbf{let} \ r = \mathbf{sample}(\text{exp}); & \quad \mathbf{snd}(\text{c}) \cdot \mathbf{fst}(\text{c})^{-sk} \\
\quad (sk, \ g^{sk}) & \quad (g^r, \ m \cdot pk^r) &
\end{array}$$

Fig. 2. Example: Algorithms of the ElGamal cryptosystem defined as abbreviations.

$$\begin{array}{ll}
\mathbf{I-DDH} = \mathbf{interface} & \\
\quad \mathbf{sig} \ \text{GETA} : \text{unit} \rightarrow \text{el} \quad \mathbf{sig} \ \text{GETBC} : \text{unit} \rightarrow \text{el} \times \text{el} & \\
\\
\text{DDH}^0 = & \text{DDH}^1 = \\
\quad \mathbf{module} & \quad \mathbf{module} \\
\quad \mathbf{fun} \ \text{GETA} () = & \quad \mathbf{fun} \ \text{GETA} () = \\
\quad \quad \mathbf{let} \ a = \mathbf{sample}(\text{exp}) \ \mathbf{in} & \quad \mathbf{let} \ a = \mathbf{sample}(\text{exp}) \ \mathbf{in} \\
\quad \quad \text{mga} := \mathbf{inr}(g^a) \ \mathbf{in} & \quad \text{init} := \mathbf{inr} (); \\
\quad \quad g^a & \quad g^a \\
\\
\mathbf{fun} \ \text{GETBC} () = & \mathbf{fun} \ \text{GETBC} () = \\
\quad \mathbf{let} \ \mathbf{inr}(x) = !\text{mga} \ \mathbf{in} & \quad \mathbf{let} \ \mathbf{inr}() = !\text{init} \ \mathbf{in} \\
\quad \text{mga} := \mathbf{inl} (); & \quad \text{init} := \mathbf{inl} (); \\
\quad \mathbf{let} \ b = \mathbf{sample}(\text{exp}) \ \mathbf{in} & \quad \mathbf{let} \ b = \mathbf{sample}(\text{exp}) \ \mathbf{in} \\
\quad (g^b, \ x^b) & \quad \mathbf{let} \ c = \mathbf{sample}(\text{exp}) \ \mathbf{in} \\
& \quad (g^b, \ g^c)
\end{array}$$

Fig. 3. Example: Definition of DDH interface and games.

for each value while disregarding the state of the configuration. It is defined as $\text{steps}_n : \text{Cfg} \rightarrow \mathcal{D}(\text{Val})$.

$$\begin{aligned}
\text{steps}_0(e, \sigma) &= \text{zero} \\
\text{steps}_n(v, \sigma) &= \text{unit}(v) \\
\text{steps}_n(e, \sigma) &= \text{bind}(\text{step}(e, \sigma), \text{steps}_{n-1})
\end{aligned}$$

Finally, the probability distribution of all values of a computation is defined as taking the following limit

$$\text{steps}(e, \sigma)(v) = \lim_{n \rightarrow \infty} \text{steps}_n(e, \sigma)(v).$$

This limit always exists since it is bounded and monotone. From this construction we obtain the definition of the probability event $v \leftarrow e$, which expresses that e evaluates to v in the initial state σ_0 chosen to have the leftmost values for each base type. Existence of such a state is guaranteed by Theorem 2.

Definition 4 (Probability of value). *When e has type $\Sigma; \cdot \mid \cdot \vdash e : \tau$ for some Σ and τ define*

$$\Pr[v \leftarrow e] = \text{steps}(e, \sigma_0)(v)$$

$$\begin{array}{c}
\overline{\langle (\mathbf{rec} \ f \ x = e) \ v; \sigma \rangle \rightarrow \langle e[(\mathbf{rec} \ f \ x = e)/f][v/x]; \sigma \rangle} \\
\\
\overline{\langle \mathbf{fst}((v_1, v_2)); \sigma \rangle \rightarrow \langle v_1; \sigma \rangle} \quad \overline{\langle \mathbf{snd}((v_1, v_2)); \sigma \rangle \rightarrow \langle v_2; \sigma \rangle} \\
\\
\overline{\langle \mathbf{case} \ \mathbf{inl}(v_1) \ \mathbf{of} \ \mathbf{inl}(x_1) \Rightarrow e_1 \mid \mathbf{inr}(x_2) \Rightarrow e_2; \sigma \rangle \rightarrow \langle e_1[v_1/x_1]; \sigma \rangle} \\
\\
\overline{\langle \mathbf{case} \ \mathbf{inr}(v_2) \ \mathbf{of} \ \mathbf{inl}(x_1) \Rightarrow e_1 \mid \mathbf{inr}(x_2) \Rightarrow e_2; \sigma \rangle \rightarrow \langle e_2[v_2/x_2]; \sigma \rangle} \\
\\
\overline{\sigma(a) = v} \quad \overline{\langle a := v; \sigma \rangle \rightarrow \langle () ; \sigma, a := v \rangle} \quad \overline{v \in \alpha} \quad \overline{\langle \mathbf{sample}(\alpha); \sigma \rangle \rightarrow_{1/|\alpha|} \langle v; \sigma \rangle}
\end{array}$$

Fig. 4. Operational Semantics

F. Module Algebra

For higher-level cryptographic arguments, where we represent cryptographic systems, oracles, games, and reductions as modules, we need to introduce the algebraic properties of module composition. We start with the identity module $\text{ID}(I)$ that implements an interface I by forwarding each call to the composing module.

Definition 5 (Identity Module). *Let*

$$I = \mathbf{interface} \ \mathbf{sig} \ F_1 : \alpha_1 \rightarrow \alpha_1' \ \dots \ \mathbf{sig} \ F_n : \alpha_n \rightarrow \alpha_n'$$

be an interface. Then

$$\text{ID}(I) = \mathbf{module} \ \mathbf{fun} \ F_1 x = F_1(x) \ \dots \ \mathbf{fun} \ F_n x = F_n(x)$$

is the identity module for interface I .

Lemma 3 (Identity module). *Assuming the interface I is well-formed, so is the identity module $\cdot ; I \vdash \text{ID}(I) : I$.*

Before we introduce sequential composition, we define an inlining operation that traverses an expression and replaces every call to a function used in one module, by the definition of the function, declared in a different module. We write this operation as $e \propto M$ and is defined to replace $F(e_1)$ in e with the corresponding function F from M applied to argument e_1 .

Definition 6 (Shared sequential composition). *Define sequential composition $(\circ) : \text{Module} \times \text{Module} \rightarrow \text{Module}$ on a pair of modules as follows*

$$\mathbf{module} \circ M_2 = \mathbf{module}$$

$$(M_1 \mathbf{val} \ f := e_1) \circ M_2 = (M_1 \circ M_2) \mathbf{val} \ f := e_1 \propto M_2$$

To model state-separating proofs adequately, we expect that all calls in the left module are resolved by definitions from the right module. This principle is captured by the following derivable typing rule.

Lemma 4 (Well-typed sequential composition). *Given modules M_1, M_2 with $\Sigma; I_1 \vdash M_1 : E$ and $\Sigma; I_2 \vdash M_2 : I_1$ we derive $\Sigma; I_2 \vdash M_1 \circ M_2 : E$.*

The interface I_1 is the connection point between M_1 and M_2 . It ensures that every call in M_1 has a corresponding

implementation in M_2 . Note that they all must be typeable in the same state Σ . Separated sequential composition based on nominals that we introduce in Section IV does not have this limitation.

In cases where the import interface does not match, we may need to weaken the imports (expressed by $I' > I$) given by the following lemma.

Lemma 5 (Weakening). *Given a module M with $\Sigma; I \vdash M : E$ and $I' > I$ then $\Sigma; I' \vdash M : E$.*

Parallel composition concatenates the functions of two modules into one bigger module and is defined as follows.

Definition 7 (Shared parallel composition). *Define parallel composition $(|) : \text{Module} \times \text{Module} \rightarrow \text{Module}$ on a pair of modules as follows*

$$\mathbf{module} \mid M_2 = M_2$$

$$(M_1 \mathbf{val} \ f := e) \mid M_2 = (M_1 \mid M_2) \mathbf{val} \ f := e$$

We can also derive a type for parallel composition of modules; however, we need to disallow cases where identifiers defined by the modules overlap. Otherwise, the export interface is not well-formed.

Lemma 6 (Well-typedness of parallel composition). *Given modules M_1, M_2 and export interfaces E_1, E_2 where the identifiers declared in E_1 are disjoint from the identifiers declared in E_2 . If $\Sigma; I \vdash M_1 : E_1$ and $\Sigma; I \vdash M_2 : E_2$ we can derive $\Sigma; I \vdash (M_1 \mid M_2) : E_1, E_2$.*

The following equations allow us to work with the modules reasoning with adversaries, deriving advantages, building oracles, and conducting proofs by game hopping. With these rules we can algebraically manipulate the module expressions to isolate certain parts for reduction.

Lemma 7 (Identities). *For a module M with type $\Sigma; I \vdash M : E$, the following equations hold.*

$$\begin{array}{ll}
\text{ID}(E) \circ M = M & \text{ID}(\mathbf{interface}) \mid M = M \\
M \circ \text{ID}(I) = M & M \mid \text{ID}(\mathbf{interface}) = M
\end{array}$$

Lemma 8 (Associativity). *For modules $M_1, M_2, M_3 \in \text{Module}$ the sequential and parallel composition operators are associative.*

$$\begin{aligned}(M_1 \circ M_2) \circ M_3 &= M_1 \circ (M_2 \circ M_3) \\ (M_1 \mid M_2) \mid M_3 &= M_1 \mid (M_2 \mid M_3)\end{aligned}$$

Lemma 9 (Interchange). *For modules M_1, M_2, M_3, M_4 where $\Sigma; I_1 \vdash M_1 : E_1$, $\Sigma; I_2 \vdash M_2 : E_2$, $\Sigma; I \vdash M_3 : I_1$, $\Sigma; I \vdash M_4 : I_2$, and I_1 defines different identifiers from I_2 , then*

$$(M_1 \mid M_2) \circ (M_3 \mid M_4) = (M_1 \circ M_3) \mid (M_2 \circ M_4)$$

The well-typedness assumptions for interchange ensure that there are no calls from M_1 to M_4 and from M_2 to M_3 .

Looking back at Lemmas 4 and 6, states are shared between modules M_1 and M_2 , which is expressed by the use of the same heap type Σ in both typing derivations: $\Sigma; I_1 \vdash M_1 : E$ and $\Sigma; I_2 \vdash M_2 : I_1$. It is this sharing that motivates the rest of the paper. In case that we compose an adversary (of export interface I-ADV) with existing modules DDH^0 and DDH^1 in Example 3, we must prevent the adversary from reading from state variables `mga` or `init`, otherwise it could trivially distinguish the games.

The alternative to having one Σ as heap-type, is to use two different heap types Σ_1 and Σ_2 , for modules M_1 and M_2 , respectively, so that $\Sigma_1; I_1 \vdash M_1 : E$ and $\Sigma_2; I_2 \vdash M_2 : I_1$. When composing M_1 and M_2 (either sequentially or in parallel), the resulting module M will be well-typed in $\Sigma_1, \Sigma_2; I_2 \vdash M : I_1$. If we do this, however, we need to worry about variable capture. This can be avoided by using separated concatenation of heaps $\Sigma_1 * \Sigma_2$ which prevents variable capture as we discuss in the next section.

Our solution is to ensure separation between modules using nominal sets.

III. NOMINALS

Nominal-sets for reasoning about open terms with names were discovered in the seminal paper by Gabbay and Pitts [10] and applied to model languages with binding. α -conversion, which is a conceptually simple but notoriously annoying problem when modeling languages with binding operators, can be seen as an instance of a nominal set, where atoms are automatically renamed to keep the naming of two terms disjoint from one another. The central idea of nominal sets is a permutation model of set theory with names (also known as Fraenkel and Mostowski sets). In this section we briefly review the nominal techniques and their properties following [15], keeping in mind that we apply these techniques to heaps in the theory of state-separating proofs. In the following section we show how to keep the internal state of modules disjoint from one another using nominal sets. We introduce Perm $\mathbb{A}$ -sets in Section III-A, equivariance in Section III-B, support sets in Section III-C, α -equivalence in Section III-D. The proofs of important lemmas and theorems presented in this section are summarized in Appendix A.

A. Perm $\mathbb{A}$ -sets

Recall the definition of atoms $\mathbb{A}$ from Definition 1. Permutations among atoms are central to nominal sets.

Definition 8 (Permutations). *Let Perm $\mathbb{A}$ denote the group of permutations that act on a finite subset of $\mathbb{A}$ and leave the rest unchanged. Perm $\mathbb{A}$ defines the usual group structure: For two permutations $\pi_1, \pi_2 \in \text{Perm } \mathbb{A}$, we write their group product as $\pi_1 \pi_2$, while π_1^{-1} denotes the inverse permutation of π_1 . Finally, we take id to denote the neutral element.*

The following definition gives us a general notion of sets that support applying a permutation of atoms to its elements.

Definition 9 (Perm $\mathbb{A}$ -set). *A set X with a group action $(\cdot) : \text{Perm } \mathbb{A} \times X \rightarrow X$ is called a Perm $\mathbb{A}$ -set (read: permutation set). The group action must obey the following equations for all $\pi_1, \pi_2 \in \text{Perm } \mathbb{A}$ and $x \in X$*

$$\text{id} \cdot x = x, \quad (\text{Identity})$$

$$\pi_1 \cdot \pi_2 \cdot x = (\pi_1 \pi_2) \cdot x. \quad (\text{Compatibility})$$

Equations (Identity) and (Compatibility) are standard for group actions. We illustrate the concept of permutation sets by a few examples to give the reader a chance to get acquainted with them.

Lemma 10 (Perm $\mathbb{A}$ -sets). *It is easy to see that the following sets are Perm $\mathbb{A}$ -sets by checking (Identity) and (Compatibility).*

- *The set $\mathbb{A}$ of atoms is a Perm $\mathbb{A}$ -set with group action $\pi \cdot a = \pi(a)$.*
- *Given Perm $\mathbb{A}$ -sets X and Y , their Cartesian product $X \times Y$ is a Perm $\mathbb{A}$ -set with group action $\pi \cdot (x, y) = (\pi \cdot x, \pi \cdot y)$.*
- *The set of real numbers $\mathbb{R}$ is a Perm $\mathbb{A}$ -set with group action $\pi \cdot r = r$.*
- *The set $P_{\text{fin}}(X) = \{F \mid F \subset X \text{ and } F \text{ finite}\}$ for any Perm $\mathbb{A}$ -set X is also a Perm $\mathbb{A}$ -set with group action $\pi \cdot F = \{\pi \cdot x \mid x \in F\}$ for $F \in P_{\text{fin}}(X)$.*

The following lemma shows how permutations act on heaps.

Lemma 11. *Heap and State are Perm $\mathbb{A}$ -sets with group action*

$$\pi \cdot (\Sigma, a : \alpha) = (\pi \cdot \Sigma, \pi(a) : \alpha)$$

$$\pi \cdot (\sigma, a := v) = (\pi \cdot \Sigma, \pi(a) := v)$$

In the last equation, v is invariant under the permutation, since v does not contain any atoms, as it is a sample.

B. Equivariance

We review the concept of equivariant functions. These capture the property of being name invariant in the sense that the result of an equivariant function does not depend on the specific names used.

Definition 10 (Equivariant function). *A function $f : X \rightarrow Y$ between Perm $\mathbb{A}$ -sets X and Y is equivariant if for all $\pi \in \text{Perm } \mathbb{A}$ and $x \in X$, $\pi \cdot f(x) = f(\pi \cdot x)$.*

In contrast, if a non-equivariant function f were to create a new atom, it is easy to construct a permutation, so that $\pi \cdot f(x) \neq f(\pi \cdot x)$.

Example 4. The function $f : \mathbb{A} \rightarrow \mathbb{A} \times \mathbb{A}$ given by $f(a) = (a, a)$ is equivariant, since for all $\pi \in \text{Perm } \mathbb{A}$ and $a \in \mathbb{A}$,

$$\pi \cdot f(a) = \pi \cdot (a, a) = (\pi \cdot a, \pi \cdot a) = f(\pi \cdot a)$$

On the other hand, the function $g : \mathbb{A} \rightarrow \mathbb{A} \times \mathbb{A}$ given by $g(a) = (a_1, a)$ is not equivariant, since for $\pi = (a_1 \ a_2)$ and some $a \in \mathbb{A}$ different from a_1 and a_2 , we have that

$$(a_1 \ a_2) \cdot g(a) = (a_2, a) \neq (a_1, a) = g((a_1 \ a_2) \cdot a)$$

Lemma 12 (Equivariant set operations). *Intersection and union are equivariant on $P_{\text{fin}}(X)$ for a $\text{Perm } \mathbb{A}$ -set X . That is, for all $A, B \in P_{\text{fin}}(X)$ and $\pi \in \text{Perm } \mathbb{A}$ it holds that $\pi \cdot (A \cap B) = (\pi \cdot A) \cap (\pi \cdot B)$ and $\pi \cdot (A \cup B) = (\pi \cdot A) \cup (\pi \cdot B)$.*

From this fact we can infer that subset inclusion and set disjointness are also equivariant in the sense that $A \subseteq B$ if and only if $\pi \cdot A \subseteq \pi \cdot B$ and $A \cap B = \emptyset$ if and only if $\pi \cdot A \cap \pi \cdot B = \emptyset$.

Finally, we prove that shared module compositions are equivariant.

Theorem 13 (Equivariance of shared module compositions). *$M_1 \circ M_2$ and $M_1 \mid M_2$ are equivariant.*

C. Support

We turn to the finiteness criterion of what is called the support of a nominal set.

Definition 11 (Finite support). *Let X be a $\text{Perm } \mathbb{A}$ -set and $S \in P_{\text{fin}}(\mathbb{A})$ a finite subset of atoms. We say S is a support set for $x \in X$ if for all permutations $\pi \in \text{Perm } \mathbb{A}$ so that $\pi(a) = a$ for all $a \in S$, it holds that $\pi \cdot x = x$.*

Finding a support set for an element in a $\text{Perm } \mathbb{A}$ -set guarantees that any permutation that preserves the atoms in the support set will also preserve the value. In some sense the support set contains at least all of the atoms that are *present* in the element, but we define this without having to say what an atom being *present* means. We expand this concept to a whole $\text{Perm } \mathbb{A}$ -set with the definition of a nominal set.

Definition 12 (Nominal set). *A $\text{Perm } \mathbb{A}$ -set X has a finite support function $\text{supp} : X \rightarrow P_{\text{fin}}(\mathbb{A})$ if for all $x \in X$, $\text{supp}(x)$ is a support set for x and for any other support set S for x , it is the case that $\text{supp}(x) \subseteq S$. A $\text{Perm } \mathbb{A}$ -set with a finite support function is called a nominal set.*

Henceforth, we will use the names X, Y and Z to denote arbitrary nominal sets. The $\text{Perm } \mathbb{A}$ -sets that we have seen until this point are also nominal sets.

Lemma 14 (Nominal sets). *The following $\text{Perm } \mathbb{A}$ sets are nominal sets.*

- The set $\mathbb{A}$ has $\text{supp}(a) = \{a\}$.
- The set $X \times Y$ has $\text{supp}(x, y) = \text{supp}(x) \cup \text{supp}(y)$.

- The set $\mathbb{R}$ has $\text{supp}(r) = \emptyset$. In fact, every $\text{Perm } \mathbb{A}$ -set with $\pi \cdot x = x$ has an empty support set and is called a discrete nominal set.
- The set $P_{\text{fin}}(X)$ has $\text{supp}(S) = \bigcup_{x \in S} \text{supp}(x)$

Lemma 15 (Heaps and states are nominal sets). *The following $\text{Perm } \mathbb{A}$ sets are nominal sets.*

- The set *Heap* has $\text{supp}(\Sigma, a : \alpha) = \text{supp}(\Sigma) \cup \{a\}$.
- The set *State* has $\text{supp}(\sigma, a := v) = \text{supp}(\sigma) \cup \{a\}$.

Note that the supp function maps an element from any nominal set into a finite set of atoms. This allows us to state disjointness generically.

Definition 13 (Disjoint support). *We say that $x \in X$ and $y \in Y$ are disjoint written $x \# y$ when $\text{supp}(x) \cap \text{supp}(y) = \emptyset$.*

Lemma 16 (Disjoint support properties). *For elements $x \in X$, $y \in Y$ and $z \in Z$ and equivariant function $f : X \rightarrow Z$ it holds that*

- if $x \# y$ then $y \# x$,
- if $x \# x$ then $\text{supp}(x) = \emptyset$,
- if $x \# y$ then $f(x) \# y$,
- if $x \# y$ then $\pi \cdot x \# \pi \cdot y$ for $\pi \in \text{Perm } \mathbb{A}$.

Now we understand what disjointness means, but we have not proposed a way to render two elements of nominal sets disjoint, which we achieve by active renaming using the permutation *fresh*.

Definition 14 (Fresh). *Fix some bijection between the atoms and the natural numbers $\mathbb{N} = \{1, 2, \dots\}$ called $\text{idx} : \mathbb{A} \rightarrow \mathbb{N}$. Define $\text{fresh} : X \times Y \rightarrow \text{Perm } \mathbb{A}$ so that³*

$$\text{fresh}(x, y)(a) = \text{idx}^{-1}(\text{idx}(a) + k(x)) \text{ for } a \in \text{supp}(y),$$

where

$$k(x) = \max_{a' \in \text{supp}(x)} \text{idx}(a').$$

The purpose of *fresh* is to choose an element $y' \in Y$ related to $y \in Y$, so that the support of a fixed $x \in X$ is disjoint from the support of the chosen y' . This property is captured in the following lemma.

Lemma 17 (Fresh disjoint). *For all elements $x \in X$ and $y \in Y$, it holds that $x \# \text{fresh}(x, y) \cdot y$.*

To show the usefulness of *fresh* we introduce separated concatenation of heaps, denoted by $\Sigma_1 * \Sigma_2$, as motivated at the end of Section II-F.

Definition 15 (Separated concatenation). *For $\Sigma_1, \Sigma_2 \in \text{Heap}$ define separated concatenation so that*

$$\Sigma_1 * \Sigma_2 = \Sigma_1, \text{fresh}(\Sigma_1, \Sigma_2) \cdot \Sigma_2.$$

Separated concatenation enforces support separation by actively renaming atoms in the right argument using *fresh*. We will re-use this pattern when defining separated compositions.

³The definition is injective, so we obtain a permutation by mapping elements from the finite set $\text{fresh}(x, y)(\text{supp}(y)) \setminus \text{supp}(y)$ to the finite set of equal size $\text{supp}(y) \setminus \text{fresh}(x, y)(\text{supp}(y))$ injectively.

D. α -equivalence

To capture the nice properties of separated concatenation, we introduce α -equivalence. Two elements are α -equivalent, if they are equal modulo a permutation of atoms. This notion of α -equivalence is defined for any nominal set.

Definition 16 (α -equivalence). *For elements $x, x' \in X$ we say that x and x' are α -equivalent written $x \equiv x'$ when there exists a permutation $\pi \in \text{Perm } \mathbb{A}$ so that $\pi \cdot x = x'$.*

Theorem 18. *α -equivalence is an equivalence relation.*

When concatenating two heaps, without considering α -equivalence, state variable capture might be unavoidable. When concatenating two heaps using separated concatenation, which takes α -renaming into account, variable capture will be avoided. α -congruence expresses the fact that we can replace heaps by α -equivalent heaps and still avoid variable capture.

Definition 17 (α -congruence). *Assuming $X_1, \dots, X_n$, and Y are nominal sets. An n -ary function $f : X_1 \times \dots \times X_n \rightarrow Y$ is an α -congruence if whenever $x_i \equiv x'_i$ for $i \in \{1, \dots, n\}$ then $f(x_1, \dots, x_n) \equiv f(x'_1, \dots, x'_n)$.*

We show that any unary equivariant function is also an α -congruence.

Example 5. *Let $f : X \rightarrow Y$ be given and assume that f is equivariant. For $x, x' \in X$ assume that $x \equiv x'$, thus there exists π so that $\pi \cdot x = x'$. Hence, $f(x') = f(\pi \cdot x) = \pi \cdot f(x)$, so $f(x) \equiv f(x')$ i.e. f is an α -congruence.*

This argument does apply to binary functions, since the support of the two elements may overlap. Swapping elements in the overlap for one argument will result in a different sharing of names; thus the structure of atoms is not preserved in the result. Note that the shared compositions are not α -congruent for this reason. However, separated concatenation is an α -congruence because state variable capture is impossible.

Theorem 19 (Separated concatenation is an α -congruence). *When $\Sigma_1 \equiv \Sigma'_1$ and $\Sigma_2 \equiv \Sigma'_2$, then $\Sigma_1 * \Sigma_2 \equiv \Sigma'_1 * \Sigma'_2$.*

IV. NOMINAL STATE-SEPARATING PROOFS

We will now demonstrate how to extend the language of state-separating proofs introduced in Section II by nominals, which results in a concept called *nominal state-separating proofs*. We argue that all concepts introduced in that section are nominal sets where atoms are state variables. We describe *nominal expressions* in Section IV-A and *nominal modules* in Section IV-B. In Section IV-C, we introduce the concepts of *nominal advantage* and *nominal indistinguishability* between pairs of games. The proofs for the lemmas and theorems in this section can be found in Appendix A.

A. Nominal Expressions

Recall that we distinguish different kinds of variables: Local variables are declared within each function, state variables – or atoms – are declared globally and shared across functions, and

identifiers are used to refer to imported functions. When moving to nominal expressions, we move away from a single shared heap to many separate heaps, as discussed in Section III-D. As a starting point, we show that $Expr$ is a $\text{Perm } \mathbb{A}$ -set.

Definition 18 (Group action for $Expr$). *Let $\pi \in \text{Perm } \mathbb{A}$ be a permutation and $e \in Expr$. We define the group action $\cdot : \text{Perm } \mathbb{A} \times Expr \rightarrow Expr$. We only present the base cases here, leaving it to the reader to define the remaining cases by forming the congruence closure. We omit the proofs of identity and compatibility.*

$$\begin{aligned} \pi \cdot (x) &= x & \pi \cdot (F(e)) &= F(\pi \cdot e) \\ \pi \cdot () &= () & \pi \cdot (\text{sample}(\alpha)) &= \text{sample}(\alpha) \\ \pi \cdot (!a) &= !\pi(a) & \pi \cdot (a := e) &= \pi(a) := (\pi \cdot e) \end{aligned}$$

Next we define the support function for $Expr$.

Definition 19 (Support function for $Expr$). *The support function $\text{supp} : Expr \rightarrow P_{\text{fin}}(\mathbb{A})$ is defined according to the structure of $Expr$. We only give base cases, since the congruence closure is straightforward and omitted in the interest of space, as is the proof of the well-definedness of the support function.*

$$\begin{aligned} \text{supp}(x) &= \emptyset & \text{supp}(F(e)) &= \text{supp}(e) \\ \text{supp}() &= \emptyset & \text{supp}(\text{sample}(\alpha)) &= \emptyset \\ \text{supp}(!a) &= \{a\} & \text{supp}(a := e) &= \{a\} \cup \text{supp}(e) \end{aligned}$$

Finally, we show that $Expr$ is a nominal set.

Lemma 20 (Nominal expressions). *The set of expressions $Expr$, together with the group action $\cdot : \text{Perm } \mathbb{A} \times Expr \rightarrow Expr$ and the support function $\text{supp} : Expr \rightarrow P_{\text{fin}}(\mathbb{A})$ forms a nominal set.*

We have already seen that $Heap$ is also a nominal set, so we state and prove that well-typedness is preserved under permutation.

Lemma 21 (Type preserved by permutation). *Given $\Sigma; I \mid e : \tau$ we have $\pi \cdot \Sigma; I \mid \pi \cdot e : \tau$.*

Moving from the static to the dynamic semantics, we show that operational semantics introduced in Section II-D is also equivariant, which means that the probability distribution on the resulting values is invariant under the group action: $\pi \cdot \Pr[v \leftarrow e] = \Pr[\pi \cdot v \leftarrow \pi \cdot e]$. We structure the argument into smaller steps, starting with the fact that being a value is preserved under permutation.

Lemma 22 (Preservation of values). *If $v \in Val$, then $\pi \cdot v \in Val$.*

Recall that the probabilistic small-step operational semantics enabled us to define a step $: Cfg \rightarrow \mathcal{D}(Cfg)$ function, that maps configurations to a probability distribution of configurations. Recall that configurations are defined as products of two nominal sets, which means that they are also nominal sets. We can show that the step-function is equivariant, meaning that the probability distribution remains intact under permutations.

With the group action derived from the group action of the configurations, we can show that step is equivariant.

Lemma 23 (Step equivariance). *The function $\text{step} : Cfg \rightarrow \mathcal{D}(Cfg)$ is equivariant.*

In Definition 3, we introduced the sub-distribution monad in order to iterate the single-step reductions of the operational semantics. We can show that the sub-distribution monad inherits the nominal set property from the nominal expressions, by the virtue that unit, bind and zero are bound to value (of functional type), for which we have already established in Lemma 20 that they are nominal sets.

Lemma 24 (Sub-Distribution equivariance). *The functions unit, bind and zero are equivariant.*

Based on these observations, we can generalize the equivariance property of the single-step relation step to the multi-step case steps.

Theorem 25 (steps equivariance). *The function $\text{steps} : Cfg \rightarrow \mathcal{D}(Cfg)$ is equivariant.*

This means that the mapping of configurations into the sub-distribution monad is equivariant, and it follows directly that the probability distribution on result values is also equivariant.

Corollary 26 (Pr equivariance). $\text{Pr}[\cdot \leftarrow \cdot] : Val \times Expr \rightarrow \mathbb{R}$ is equivariant.

B. Nominal Modules

Recall from Section II-F that two modules can be composed either sequentially, which means that calls to external functions are resolved by inlining, or in parallel, which means that the functions of either module are merged. In this section we show that modules form a nominal set, and that we can define separating module composition based on the results of preceding sections.

As usual, as the first step, we define a group action for modules, for which we write $(\cdot) : \text{Perm } \mathbb{A} \times Module \rightarrow Module$. Its definition is straightforward.

Definition 20 (Group action for *Module*).

$$\begin{aligned} \pi \cdot \text{module} &= \text{module} \\ \pi \cdot (M \text{ fun } F \ x = e) &= ((\pi \cdot M) \text{ fun } F \ x = (\pi \cdot e)) \end{aligned}$$

Note that F is a function identifier, and x is a local variable; thus the permutation does not affect them. We omit the well-formedness proof of the group action. As a second step, we define the support function $\text{supp} : Module \rightarrow P_{\text{fin}}(\mathbb{A})$ for modules.

Definition 21 (Support function for *Module*).

$$\begin{aligned} \text{supp}(\text{module}) &= \emptyset \\ \text{supp}(M \text{ fun } F \ x = e) &= \text{supp}(M) \cup \text{supp}(e) \end{aligned}$$

We omit the well-formedness proof of the support function.

Lemma 27 (Nominal modules). *The set *Module* forms a nominal set together with the group action $\cdot : \text{Perm } \mathbb{A} \times Module \rightarrow Module$ and the support function $\text{supp} : Module \rightarrow P_{\text{fin}}(\mathbb{A})$.*

The main advantage of introducing nominal modules is that users do not have to worry about constructing renaming permutations when composing modules using separated composition as described next. The respective permutations are automatically generated by the theory.

Definition 22 (Separated composition). *Define separated sequential composition as*

$$M \otimes N = M \circ (\text{fresh}(M, N) \cdot N),$$

and parallel composition as

$$M \parallel N = M \mid (\text{fresh}(M, N) \cdot N).$$

Note the similarity of how we apply fresh here in relation to how we define separating concatenation of heaps in Section III-C.

To express the heap of separated composition, we introduce a pruning operation. To type a separated composition of modules M_1 and M_2 , we need to ensure that the separated concatenation of heaps $\Sigma_1 * \Sigma_2$ are aligned. If there are atoms in Σ_1 that do not appear in M_1 , then the permutations applied internally may not match. To rectify this problem, we define a *pruning* operation that strengthens Σ_1 to Σ'_1 which only contains atoms that are also used in M_1 . We write $\Sigma'_1 = \text{prune}(\Sigma_1, M_1)$ for pruning.

Lemma 28 (Well-typedness of separated sequential composition). *Given modules M_1, M_2 with $\Sigma_1; I_1 \vdash M_1 : E$ and $\Sigma_2; I_2 \vdash M_2 : E$ we can derive $\Sigma_3; I_2 \vdash M_1 \otimes M_2 : E$ where $\Sigma_3 = \text{prune}(\Sigma_1, M_1) * \Sigma_2$.*

Lemma 29 (Well-typedness of separated parallel composition). *Given modules M_1, M_2 and export interfaces E_1, E_2 where the identifiers declared in E_1 are disjoint from the identifiers declared in E_2 . If $\Sigma_1; I \vdash M_1 : E_1$ and $\Sigma_2; I \vdash M_2 : E_2$ we can derive $\Sigma_3; I \vdash (M_1 \parallel M_2) : E_1, E_2$ where $\Sigma_3 = \text{prune}(\Sigma_1, M_1) * \Sigma_2$.*

As a consequence of the separation and the fact that shared compositions are equivariant, functions we are able to derive the following congruence theorem.

Theorem 30 (Separated congruence). *For modules where $M_1 \equiv M_1'$ and $M_2 \equiv M_2'$ we have*

$$\begin{aligned} M_1 \otimes M_2 &\equiv M_1' \otimes M_2', \\ M_1 \parallel M_2 &\equiv M_1' \parallel M_2'. \end{aligned}$$

We now show that the desired algebraic rules are preserved under either α -equivalence or equality.

Lemma 31 (Identities). *For a module M with type $\Sigma; I \vdash M : E$, the following identities hold.*

$$\begin{aligned} \text{ID}(E) \otimes M &= M & \text{ID}(\text{interface}) \parallel M &= M \\ M \otimes \text{ID}(I) &= M & M \parallel \text{ID}(\text{interface}) &= M \end{aligned}$$

Lemma 32 (Associativity). *For modules M_1, M_2, M_3 the separated sequential and parallel composition operators are associative up to α -equivalence.*

$$(M_1 \otimes M_2) \otimes M_3 \equiv M_1 \otimes (M_2 \otimes M_3)$$

$$(M_1 \parallel M_2) \parallel M_3 \equiv M_1 \parallel (M_2 \parallel M_3)$$

We lose the strict equality of the associativity rules; however, as we will see, this is sufficient to be compatible with nominal indistinguishability. Finally, we prove that also interchange holds under α -equivalence.

Lemma 33 (Interchange). *For modules M_1, M_2, M_3, M_4 where $\Sigma_1; I_1 \vdash M_1 : E_1$, $\Sigma_2; I_2 \vdash M_2 : E_2$, $\Sigma_3; I_3 \vdash M_3 : I_1$, $\Sigma_4; I_4 \vdash M_4 : I_2$, and I_1 defines different identifiers from I_2 , then*

$$(M_1 \parallel M_2) \otimes (M_3 \parallel M_4) \equiv (M_1 \otimes M_3) \parallel (M_2 \otimes M_4)$$

C. Nominal Indistinguishability

In cryptography, the security of a cryptographic protocol is captured by an adversary being able to distinguish between two games, M_1 and M_2 . After observing an exchange of messages (also called a transcript) that a verifier and a prover play together, the task of the adversary is to guess which of the two games was played. If the cryptographic protocol is insecure, the adversary can identify with non-negligible probability the correct game that was played. In our setting, following [6], games and adversaries can be constructed by defining and composing modules.

Definition 23 (Game). *An E-game is a module G with type $\Sigma; \cdot \vdash G : E$ for any Σ , i.e. it cannot contain any calls $F(e)$ to imported functions.*

As an example, consider the DDH games from Example 3. Both DDH^0 and DDH^1 are I-DDH-games.

Example 6. *As part of our running example, we now introduce the public key one time secrecy games that are presented along with their interface in Figure 5. We leave it to the reader to verify that*

$$\text{mpk} : \text{unit} + \text{el}, \text{flag} : \text{bool}; \cdot \vdash \text{PK-OTS}^0 : \text{I-PK-OTS\$}$$

$$\text{mpk} : \text{unit} + \text{el}, \text{flag} : \text{bool}; \cdot \vdash \text{PK-OTS}^1 : \text{I-PK-OTS\$}$$

Thus, both modules are I-PK-OTS\\$-games. As a shorthand we refer to the game pair as $\text{PK-OTS\$} = (\text{PK-OTS}^0, \text{PK-OTS}^1)$.

An adversary is modeled as a module as well, which exports a single method called RUN. Recall that we introduced the I-ADV for the adversary, introduced in Example 2.

Definition 24 (Adversary). *An E-adversary is a module $\mathcal{A}$ with type $\Sigma; E \vdash \mathcal{A} : \text{I-ADV}$ for any Σ .*

With definitions for games and adversaries in place we define advantage to be the difference in behavior of the adversary given the two games.

I-PK-OTS\\$ = interface
sig GET : unit $\rightarrow$ el **sig** QUERY : el $\rightarrow$ el $\times$ el

PK-OTS\\$⁰ =
module
fun GET () =
 let inl () = !mpk in
 let pk = snd(keygen) in
 mpk := inr(pk);
 pk

PK-OTS\\$¹ =
module
fun GET () =
 let inl () = !mpk in
 let pk = snd(keygen) in
 mpk := inr(pk);
 pk

fun QUERY m =
 let inr(pk) = !mpk in
 let inl () = !flag in
 flag := inr ();
 enc pk m

fun QUERY m =
 let inr(pk) = !mpk in
 let inl () = !flag in
 flag := inr ();
 sample(el $\times$ el)

Fig. 5. Example: Definition of games PK-OTS\\$ specialized for ElGamal.

Definition 25 (Advantage). *We define an E-adversary $\mathcal{A}$'s advantage to distinguish between E-games G_1, G_2 as*

$$\text{Adv}_{G_1, G_2}(\mathcal{A}) = |\Pr[\text{true} \leftarrow \text{RUN}() \times (\mathcal{A} \otimes G_1)] - \Pr[\text{true} \leftarrow \text{RUN}() \times (\mathcal{A} \otimes G_2)]|$$

From this definition we can easily derive symmetry

$$\text{Adv}_{G_1, G_2}(\mathcal{A}) = \text{Adv}_{G_2, G_1}(\mathcal{A}),$$

and the triangle inequality

$$\text{Adv}_{G_1, G_3}(\mathcal{A}) \leq \text{Adv}_{G_1, G_2}(\mathcal{A}) + \text{Adv}_{G_2, G_3}(\mathcal{A}).$$

With these properties, through a game hopping argument, we can bound the adversary's advantage of distinguishing between a pair of games representing the security property.

Finally, based on this notion of advantage, we introduce the notion of perfect indistinguishability between two games.

Definition 26 (Perfect Indistinguishability). *E-games G_1 and G_2 are said to be perfectly indistinguishable written $G_1 \approx_0 G_2$ when*

$$\text{Adv}_{G_1, G_2}(\mathcal{A}) = 0$$

for all E-adversaries $\mathcal{A}$.

In contrast to related work and thanks to the use of nominals, our version of perfect indistinguishability is based on separated composition, rendering any consideration regarding disjointness assumptions entirely unnecessary. To our knowledge, this is a significant improvement of other existing works on state-separating proofs [6], [8], [13]. The elegance of our approach shines through in our running example: No disjointness assumptions are necessary.

Example 7. *This example demonstrates, how to express PK-OTS\\$ in terms of DDH using perfect indistinguishability. For this, we define a reduction*

RED = **module**

```

fun GET () =           fun QUERY m =
  let inl () = !stop;    let rsh := GETBC();
  stop := inr ();       (fst (rsh), m · snd(rhs))
GETA()

```

that is well-typed $\text{stop} : \text{unit} + \text{unit}$; $\text{I-DDH} \vdash \text{RED} : \text{I-PK-OTS}$. We show that $\text{PK-OTS}^0 \approx_0 \text{RED} \otimes \text{DDH}^0$ and $\text{PK-OTS}^1 \approx_0 \text{RED} \otimes \text{DDH}^1$.

Let an I-PK-OTS-adversary $\mathcal{A}$ be given. To show that

$$\text{Adv}_{\text{PK-OTS}^0, \text{RED} \otimes \text{DDH}^0}(\mathcal{A}) = 0$$

we prove that the functions GET and QUERY exhibit the same sub-distribution of answers for each of the two modules.

This is proven formally in a probabilistic relational Hoare logic (see Section V). In the interest of space we will resort to the following argument.

We define an invariant to act as the relation between the state of the two games. As long as the support of the relation is limited to the support of the games, the adversary cannot break it, as the atoms of the games are inaccessible to the adversary due to separated composition. In this proof, the relation encodes the fact that matches of **inl** and **inr** will have the same outcome, and that under certain conditions **mpk** and **mga** contain the same value.

Likewise, we formally prove that

$$\text{Adv}_{\text{PK-OTS}^1, \text{RED} \otimes \text{DDH}^1}(\mathcal{A}) = 0.$$

Next, we confirm what we would expect to hold.

Theorem 34 (α -equivalence implies perfect indistinguishability). For all E-games G_1, G_2 , when $G_1 \equiv G_2$, then $G_1 \approx_0 G_2$.

As the main result, we show that advantage respects perfect indistinguishability. Note that advantage between games is real-valued in the interval $[0, 1]$.

Theorem 35 (Advantage congruence). For E-adversaries $\mathcal{A}$ and $\mathcal{A}'$, so that $\mathcal{A} \approx_0 \mathcal{A}'$ and E-games G_1, G_1', G_2, G_2' where $G_1 \approx_0 G_1'$ and $G_2 \approx_0 G_2'$.

$$\text{Adv}_{G_1, G_2}(\mathcal{A}) = \text{Adv}_{G_1', G_2'}(\mathcal{A}')$$

This theorem is seldom used in its full generality. Usually it is used in a context where $\mathcal{A} = \mathcal{A}'$.

The consequence of this theorem is that we can freely replace perfectly indistinguishable games in a context consisting of module composition and advantage arithmetic, as in the next example. First, we show how to perform a reduction.

Theorem 36 (Reduction). For an E-adversary $\mathcal{A}$, I-games G_1, G_2 and a module R with type Σ ; $\text{I} \vdash R : E$ it holds that

$$\text{Adv}_{R \otimes G_1, R \otimes G_2}(\mathcal{A}) = \text{Adv}_{G_1, G_2}(\mathcal{A} \otimes R)$$

We finish our running example by showing the reduction.

Example 8 (PK-OTS reduction). For all I-PK-OTS-adversaries $\mathcal{A}$ it holds that

$$\text{Adv}_{\text{PK-OTS}}(\mathcal{A}) = \text{Adv}_{\text{DDH}}(\mathcal{A} \otimes \text{RED})$$

Since

$$\begin{aligned}
 & \text{Adv}_{\text{PK-OTS}^0, \text{PK-OTS}^1}(\mathcal{A}) \\
 &= \text{Adv}_{\text{RED} \otimes \text{DDH}^0, \text{RED} \otimes \text{DDH}^1}(\mathcal{A}) \\
 &= \text{Adv}_{\text{DDH}^0, \text{DDH}^1}(\mathcal{A} \otimes \text{RED}),
 \end{aligned}$$

where the first equality holds by Theorem 35 and the second holds by Theorem 36.

Putting all pieces together, we can now solve the problem presented in the introduction: The triangle inequality can be easily expressed in the setting of nominal state-separating proofs. No disjointness requirements are necessary; the renaming of state variables is taken care of by the nominal nature of the operators.

Corollary 37 (Transitivity of perfect indistinguishability). Given E-games G_1, G_2, G_3 where $G_1 \approx_0 G_2$ and $G_2 \approx_0 G_3$, then $G_1 \approx_0 G_3$.

V. MECHANIZATION

We apply the theory of nominal state-separating proofs to develop Nominal-SSProve⁴ as an extension to the Rocq framework SSProve. In Nominal-SSProve, when quantifying over modules, those modules can always be assumed to be disjoint from each other. This means that when working within Nominal-SSProve, disjointness assumptions about state variables are not required in contrast to other implementations of non-nominal state-separating proofs. This renders formalizations of security proofs in Nominal-SSProve considerably easier.

A. Soundness Proof

The soundness of Nominal-SSProve is derived from the existing soundness of SSProve [13]. First, we show that the probabilistic interpretation of expression e is invariant under renaming of state variables $\text{Pr}[\text{true} \leftarrow e] = \text{Pr}[\text{true} \leftarrow \pi \cdot e]$ (see Corollary 26). Second, we show that when two modules have disjoint state variable sets then shared module composition as defined by SSProve is α -equivalent to separated module composition as defined by Nominal-SSProve. Therefore, when the adversary $\mathcal{A}$ is disjoint from G and G' then $\text{Adv}_{G, G'}^\#(\mathcal{A}) = \text{Adv}_{G, G'}(\mathcal{A})$. This allows us to show that the soundness of the program logic in SSProve

$$\forall \mathcal{A}. G \# \mathcal{A} \wedge G' \# \mathcal{A} \supset \text{Adv}_{G, G'}^\#(\mathcal{A}) = 0$$

entails the soundness of the same program logic used in Nominal-SSProve

$$\forall \mathcal{A}. \text{Adv}_{G, G'}(\mathcal{A}) = 0.$$

Thus, we build upon SSProve's notion of perfect indistinguishability without having to modify the soundness argument of the program logic.

⁴<https://github.com/MarkusKL/nominal-ssprove/tree/v1.0.0>

B. Example

We have mechanized the running example of this paper in Nominal-SSProve, which includes a mechanization of the PK-OTS security property, as shown in earlier sections. While developing this example we discovered two mistakes in the existing mechanization of the PK-OTS proof for ElGamal in SSProve. First, the definition of ElGamal decryption was incorrect in the source code⁵. The authors would have noticed that, if they had tried to mechanize the proof of correctness of the ElGamal cryptosystem. Second, the definition of PK-OTS is inadequate for public key encryption even after an attempt was made to fix it. The original definition given in [1, p. 6, fig. 11] is called CPA. As later discovered, this definition does not allow the adversary to obtain the public key used to encrypt the challenge message. However, the updated definition in [13, p. 13, fig. 11] and the source code of SSProve are still wrong. In those developments, key generation is part of `Challenge`, so the public key is not accessible to the adversary through `Get_pk` before choosing the argument `mes`. Giving the adversary early access to the public key is essential to complete arguments such as the reduction from many-time security to one-time security [17, Sec 15.2], which distinguishes public key encryption from symmetric encryption.

The curious reader might have noticed our non-standard formulation of the DDH game in Figure 3 in Section II-C. A standard formulation takes all two or three samples in the same function call; however, we rely on this formulation to complete the formal proof. If we could encode the fact that a value stored in the heap is randomly chosen, we could let the DDH games eagerly take all two or three samples, and let the reduction RED save the last two values until they are needed for the encryption. We leave it as future work to resolve the issue of lazy versus eager sampling in the context of state-separating proofs. The problem exists specifically for formulations in the style of state-separating proofs. If we instead retain control flow and selectively invoke the adversary as in [3], we get to use the fact that the value is randomly chosen.

As for implementation, we develop a general theory of nominal sets in Rocq using Hierarchy Builder [7]. We encode the definitions of separated composition operators in Rocq and show that they form the module algebra that we discuss in Section IV-B. When doing proofs in Nominal-SSProve, we heavily rely on Rocq’s congruence system [20] to support the proof mechanization process. In conclusion, we have encoded the entire theory discussed in this paper in Rocq, and the interested reader is invited to consult the git repository.

VI. CONCLUSION

We highlight the problem with ad-hoc disjointness assumptions in mechanizations of state-separating proofs. This is solved by introducing nominal state-separation proofs, where we rely on active renaming to enforce name separation. The benefit of nominal state-separating proofs is succinct equations used to reason about advantage between games without

considering separation. Implementing nominal state-separating proofs as Nominal-SSProve has shown actual improvements: Theorems that quantify over an adversary need not include assumptions about state separation, and rewriting using perfect indistinguishability is seamless; thus resulting in shorter proofs.

REFERENCES

- [1] Carmine Abate, Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Cătălin Hrițcu, Kenji Maillard, and Bas Spitters. Ssprove: A foundational framework for modular cryptographic proofs in coq. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, pages 1–15, 2021.
- [2] Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. Mechanized metatheory for the masses: The poplmark challenge. In Joe Hurd and Tom Melham, editors, *Theorem Proving in Higher Order Logics*, pages 50–65, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [3] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *Proceedings of the 31st Annual Conference on Advances in Cryptology, CRYPTO’11*, page 71–90, Berlin, Heidelberg, 2011. Springer-Verlag.
- [4] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. Probabilistic relational hoare logics for computer-aided security proofs. In Jeremy Gibbons and Pablo Nogueira, editors, *Mathematics of Program Construction*, pages 1–6, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [5] Chris Brzuska, Antoine Delignat-Lavaud, Christoph Egger, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. Key-schedule security for the tls 1.3 standard. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, pages 621–650, Cham, 2022. Springer Nature Switzerland.
- [6] Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. State separation for code-based game-playing proofs. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 222–249, Cham, 2018. Springer International Publishing.
- [7] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In Zena M. Ariola, editor, *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020)*, volume 167 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 34:1–34:21, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [8] François Dupressoir, Konrad Kohbrok, and Sabine Oechsner. Bringing state-separating proofs to easycript a security proof for cryptobox. In *2022 IEEE 35th Computer Security Foundations Symposium (CSF)*, pages 227–242, 2022.
- [9] T. Elgamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [10] Murdoch J. Gabbay and Andrew M. Pitts. A new approach to abstract syntax with variable binding. *Formal Aspects of Computing*, 13(3–5):341–363, 2001.
- [11] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. Asynchronous probabilistic couplings in higher-order separation logic. *Proc. ACM Program. Lang.*, 8(POPL), jan 2024.
- [12] Philipp G. Haselwarter, Kwing Hei Li, Alejandro Aguirre, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. Approximate relational reasoning for higher-order probabilistic programs. *Proc. ACM Program. Lang.*, 9(POPL), January 2025.
- [13] Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Carmine Abate, Nikolaj Sidorenko, Cătălin Hrițcu, Kenji Maillard, and Bas Spitters. Ssprove: A foundational framework for modular cryptographic proofs in coq. *ACM transactions on programming languages and systems*, 45(3):1–61, 2023.
- [14] A. K. McIver. Quantitative refinement and model checking for the analysis of probabilistic systems. In *Proceedings of the 14th International Conference on Formal Methods, FM’06*, page 131–146, Berlin, Heidelberg, 2006. Springer-Verlag.

⁵E.g. at this recent point: <https://github.com/SSProve/ssprove/tree/4d594b7>.

- [15] Andrew M. Pitts. Nominal sets. https://people.cs.nott.ac.uk/pszvc/mgs/MGS2011_nominal_sets.pdf, 2011. [Online; accessed 29-Jan-2025].
- [16] J.C. Reynolds. Separation logic: a logic for shared mutable data structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*, pages 55–74, 2002.
- [17] Mike Rosulek. The joy of cryptography, 2020. <https://joyofcryptography.com>.
- [18] Youngju Song, Minki Cho, Dongjae Lee, Chung-Kil Hur, Michael Sammler, and Derek Dreyer. Conditional contextual refinement. 7(POPL), January 2023.
- [19] Youngju Song and Dongjae Lee. Refinement composition logic. *Proceedings of the ACM on Programming Languages*, 8(ICFP), August 2024.
- [20] Matthieu; Harvard University Sozeau. A new look at generalized rewriting in type theory. *Journal of Formalized Reasoning*; Vol 2, pages No 1 (2009); 41–62, 2009.
- [21] The Coq Development Team. The coq proof assistant, September 2023. <https://doi.org/10.5281/zenodo.11551177>.

APPENDIX A OMITTED PROOFS

Proof (Theorem 13). First, we prove that inlining is equivariant i.e. $\pi \cdot (e \times M) = (\pi \cdot e) \times (\pi \cdot M)$. We proceed by induction on e and consider the $F(e)$ case with **fun** $F\ x = e'$ in M .

$$\begin{aligned} \pi \cdot (F(e) \times M) &= \pi \cdot ((\lambda x, e') (e \times M)) \\ &= (\lambda x, \pi \cdot e') (\pi \cdot (e \times M)) \\ &= (\lambda x, \pi \cdot e') ((\pi \cdot e) \times (\pi \cdot M)) \\ &= (\pi \cdot F(e)) \times (\pi \cdot M) \end{aligned}$$

We finish the proof by induction over M_1 . □

Proof (Lemma 15). We prove that $\text{supp}(\Sigma)$ is the minimal support. Let S be a support set for Σ . Assume for contradiction that $a \in \text{supp}(\Sigma) \setminus S$. Pick an $a' \notin S \cup \text{supp}(\Sigma)$ and note that $(a\ a') \cdot \Sigma \neq \Sigma$, which is shown by induction over Σ using the fact that a is in $\text{supp}(\Sigma)$. Then S cannot be a support set since neither a or a' is in S , which forces a contradiction; thus $\text{supp}(\Sigma) \setminus S = \emptyset$, which lets us conclude that $\text{supp}(\Sigma) \subseteq S$. The proof for *State* is similar. □

Proof (Lemma 17). Let $a' \in \text{supp}(x)$ and $a \in \text{supp}(y)$ be given. We have

$$\begin{aligned} \text{idx}(\text{fresh}(x, y)(a)) &= \text{idx}(a) + k(x) \\ &\geq \text{idx}(a) + \text{idx}(a') \\ &> \text{idx}(a'), \end{aligned}$$

thus $a \neq \text{fresh}(x, y)(a)$. □

Proof (Theorem 19). Assume that $\pi_1 \cdot \Sigma_1 = \Sigma'_1$ and $\pi_2 \cdot \Sigma_2 = \Sigma'_2$. Define $\pi' = \text{fresh}(\pi_1 \cdot \Sigma_1, \pi_2 \cdot \Sigma_2) \pi_2$ and extend

$$\pi(a) = \begin{cases} \pi_1(a) & \text{if } a \in \text{supp}(\Sigma_1) \\ \pi'(a) & \text{if } a \in \text{supp}(\text{fresh}(\Sigma_1, \Sigma_2) \cdot \Sigma_2) \end{cases}$$

to define a permutation. Then

$$\begin{aligned} \pi \cdot (\Sigma_1 * \Sigma_2) &= \pi \cdot \Sigma_1, \pi \cdot \text{fresh}(\Sigma_1, \Sigma_2) \cdot \Sigma_2 \\ &= \pi_1 \cdot \Sigma_1, \text{fresh}(\pi_1 \cdot \Sigma_1, \pi_2 \cdot \Sigma_2) \cdot \pi_2 \cdot \Sigma_2 \\ &= \pi_1 \cdot \Sigma_1 * \pi_2 \cdot \Sigma_2 \\ &= \Sigma'_1 * \Sigma'_2. \end{aligned}$$
□

Proof (Lemma 23). By induction on small-step derivations we can show, that $\langle e; \sigma \rangle \rightarrow_p \langle e'; \sigma' \rangle$ if and only if $\langle \pi \cdot e, \pi \cdot \sigma \rangle \rightarrow_p \langle \pi \cdot e', \pi \cdot \sigma' \rangle$. Then

$$\begin{aligned} \pi \cdot \text{step}(e, \sigma)(e', \sigma') &= \pi \cdot p \\ &= p \\ &= \text{step}(\pi \cdot e, \pi \cdot \sigma)(\pi \cdot e', \pi \cdot \sigma'). \end{aligned}$$
□

Proof (Lemma 24). We show that bind is equivariant.

$$\begin{aligned} \text{bind}(\pi \cdot p, \pi \cdot f)(\pi \cdot y) &= \sum_{x \in X} p(\pi^{-1} \cdot x) \cdot (\pi \cdot f)(x)(\pi \cdot y) \\ &= \sum_{x' \in X} p(x') \cdot (\pi \cdot f)(\pi \cdot x')(\pi \cdot y) \\ &= \sum_{x' \in X} p(x') \cdot f(x')(y) \\ &= \text{bind}(p, f)(y) \end{aligned}$$
□

Proof (Lemma 28). We weaken the heap of M_1 to obtain $\Sigma_3; I_1 \vdash M_1 : E$. We apply $\pi = \text{fresh}(M_1, \Sigma_2)$ to obtain $\pi \cdot \Sigma_2; I_2 \vdash \pi \cdot M_2 : I_1$ and weaken the heap to get $\Sigma_3; I_2 \vdash \pi \cdot M_2 : E$. Using Lemma 4 we get $\Sigma_3; I_2 \vdash M_1 \circ \pi \cdot M_2 : E$, and since $\text{supp}(M_2) \subseteq \text{supp}(\Sigma_2)$, then $\pi \cdot M_2 = \text{fresh}(M_1, M_2) \cdot M_2$, so we have derived $\Sigma_3; I_2 \vdash M_1 \otimes M_2 : E$. □

Proof (Lemma 29). Similar to proof of Lemma 29. □

Proof (Theorem 30). Similar to proof of Lemma 19. □

Proof (Lemma 31). Note that

$$\text{fresh}(\text{ID}(E), M) = \text{fresh}(\emptyset, M) = \text{id},$$

so

$$\begin{aligned} \text{ID}(E) \otimes M &= \text{ID}(E) \circ M = M \\ \text{ID}(\text{interface}) \parallel M &= \text{ID}(\text{interface}) \mid M = M \end{aligned}$$

Note that for any interface I , $\pi \cdot \text{ID}(I') = \text{ID}(I')$, so

$$\begin{aligned} M \otimes \text{ID}(I) &= M \circ \text{ID}(I) = M \\ M \parallel \text{ID}(\text{interface}) &= M \mid \text{ID}(\text{interface}) = M \end{aligned}$$
□

Proof (Lemma 32). Define

$$\begin{aligned} \pi_1 &= \text{fresh}(M_1, M_2), \\ \pi_{12} &= \text{fresh}(M_1 \otimes M_2, M_3), \\ \pi'_{12} &= \text{fresh}(\text{fresh}(M_1, M_2 \cdot M_2), M_3), \\ \pi'_1 &= \text{fresh}(M_1, M_2 \otimes M_3), \\ \pi'_2 &= \text{fresh}(M_2, M_3). \end{aligned}$$

We have

$$\begin{aligned}
(M_1 \otimes M_2) \otimes M_3 &= (M_1 \circ \pi_1 \cdot M_2) \circ \pi_{12} \cdot M_3 \\
&= M_1 \circ (\pi_1 \cdot M_2 \circ \pi_{12} \cdot M_3) \\
&\equiv M_1 \circ (\pi_1 \cdot M_2 \circ \pi'_{12} \cdot M_3) \\
&\equiv M_1 \circ (\pi'_1 \cdot M_2 \circ \pi'_1 \pi'_2 \cdot M_3) \\
&= M_1 \circ \pi'_1 \cdot (M_2 \circ \pi'_2 \cdot M_3) \\
&= M_1 \otimes (M_2 \otimes M_3)
\end{aligned}$$

The proof of associativity for parallel composition is similar. $\square$

Proof (Lemma 33). The proof of interchange is similar to that of Lemma 32, but in this case there are four separated modules. In the mechanization, these proofs are completed using light proof automation. $\square$

Proof (Theorem 34). Let E-games G_1, G_2 and an E-adversary $\mathcal{A}$ be given. Assume that $\pi \cdot G_1 = G_2$, then

$$\begin{aligned}
&\Pr[\mathbf{true} \leftarrow \text{RUN}() \propto (\mathcal{A} \otimes G_2)] \\
&= \Pr[\mathbf{true} \leftarrow \text{RUN}() \propto (\mathcal{A} \otimes \pi \cdot G_1)] \\
&= \Pr[\mathbf{true} \leftarrow \text{RUN}() \propto \pi' \cdot (\mathcal{A} \otimes G_1)] \\
&= \Pr[\mathbf{true} \leftarrow \text{RUN}() \propto (\mathcal{A} \otimes G_1)]
\end{aligned}$$

due to α -congruence for separated composition followed by equivariance, thus $\text{Adv}_{G_1, G_2}(\mathcal{A}) = 0$. $\square$

Proof (Theorem 35). We have

$$\begin{aligned}
&\text{Adv}_{G_1, G_2}(\mathcal{A}) \\
&\leq \text{Adv}_{G_1, G_1'}(\mathcal{A}) + \text{Adv}_{G_1', G_2'}(\mathcal{A}) + \text{Adv}_{G_2', G_2}(\mathcal{A}) \\
&\leq \text{Adv}_{G_1', G_2'}(\mathcal{A}),
\end{aligned}$$

and likewise $\text{Adv}_{G_1', G_2'}(\mathcal{A}) \leq \text{Adv}_{G_1, G_2}(\mathcal{A})$. $\square$

Proof (Theorem 36). Follows from associativity of separated sequential composition. $\square$

Proof (Corollary 37). Let $\mathcal{A}$ be given. By Theorem 35 we have

$$\text{Adv}_{G_1, G_3}(\mathcal{A}) = \text{Adv}_{G_2, G_2}(\mathcal{A}) = 0.$$

$\square$